TRIBHUVAN UNIVERSITY

CENTRAL DEPARTMENT OF COMPUTER SCIENCE AND INFORMATION TECHNOLOGY

CASE STUDY ON ONLINE FOOD DELIVERY SYSTEM

SUBMITTED BY:

NAME: YUJAN BASNET

ROLL NO: 18 (A)

MSC. CSIT

SUBMITTED TO:

PROF. DR. SUBARNA SHAKYA

Table Of Contents

List of Figures

1. Introduction

The Online Food Delivery System is a software application that allows customers to browse food menus, place orders through a digital platform and have it delivered to their home. The increase of e-commerce and mobile apps has made this possible and the system integrates customers, restaurants, and delivery personnel into a unified digital system. This case study focuses on how the Object-Oriented Programming concepts are used in designing and implementing the system [1].

2. Objectives

1. To apply object-oriented concepts to model a food delivery system.
2. To identify key classes, objects, and their relationships in the system.
3. To use UML diagrams to visualize the structure and behavior of the system.

3. Description of the Case Study Area

In an Online Food Delivery System, users (customers) can register, browse menus, and place orders from various restaurants. The system assigns delivery agents and processes payments. It involves:

- Real-world entities modeled as objects.
- Roles like Customer, Restaurant, FoodItem, Order, DeliveryAgent, and Admin.
- Relationships such as inheritance (User superclass), and composition (an Order contains FoodItems).

4. The Requirement Model

The major requirements for the Online Food Delivery System can be categorized in the following ways.

Functional Requirements

The major function that system most perform are:

- Register/Login (for Customer, Restaurant, Delivery Agent)
- Search restaurants and food items
- Place, update, or cancel orders
- Assign deliveries and update order status

Non-Functional Requirements

- The system should be reliable.
- The system should be scalable.
- Customer payment information should be encrypted.
- The app should be easy to use and navigate.
- Order processing should be fast.

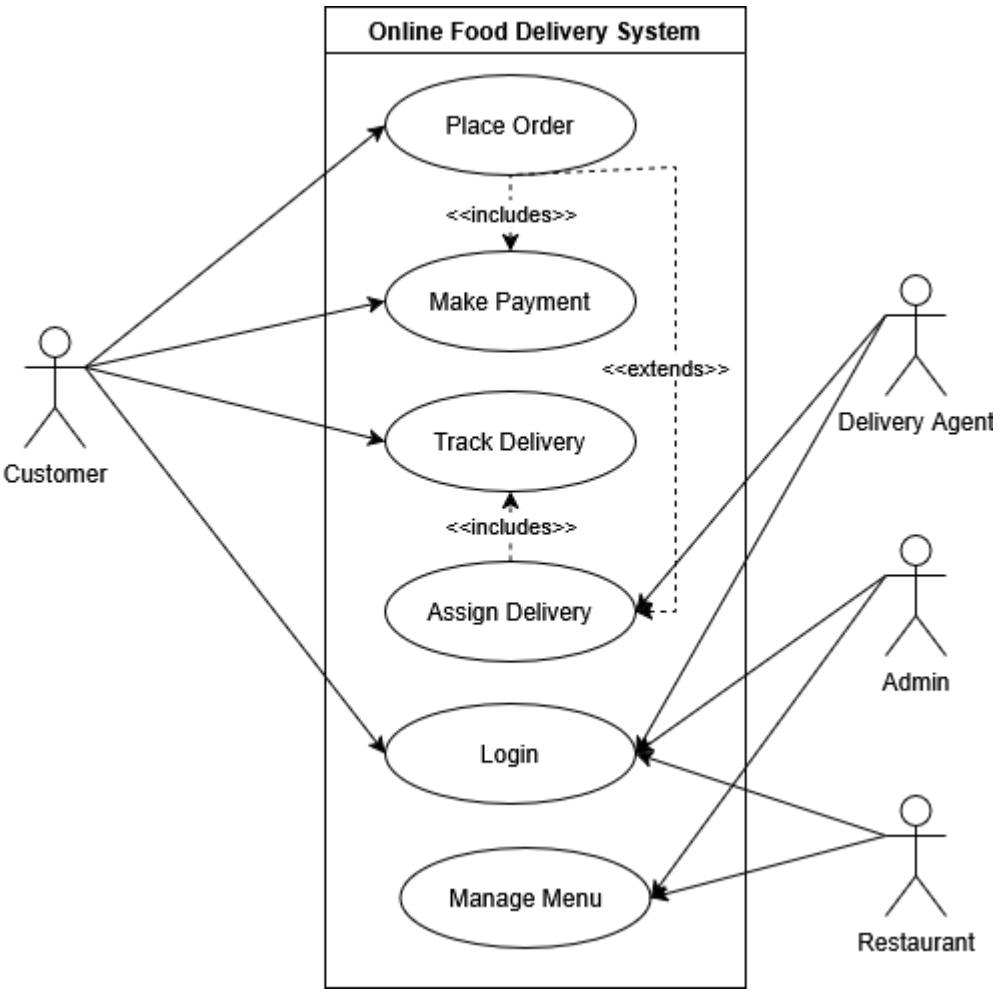

Figure 1: Use Case Diagram

5. The Analysis Model

Sequence Diagram

The Figure 2 [2] illustrates the order placement workflow in the food delivery system. The Customer initiates the process by placing an order through the OrderService, which first checks item availability with the Restaurant. If items are available, the system processes payment via PaymentService, confirms the order with the restaurant, and assigns a DeliveryAgent. Successful completion triggers an order confirmation to the customer; if items are unavailable, an order rejection is sent instead.

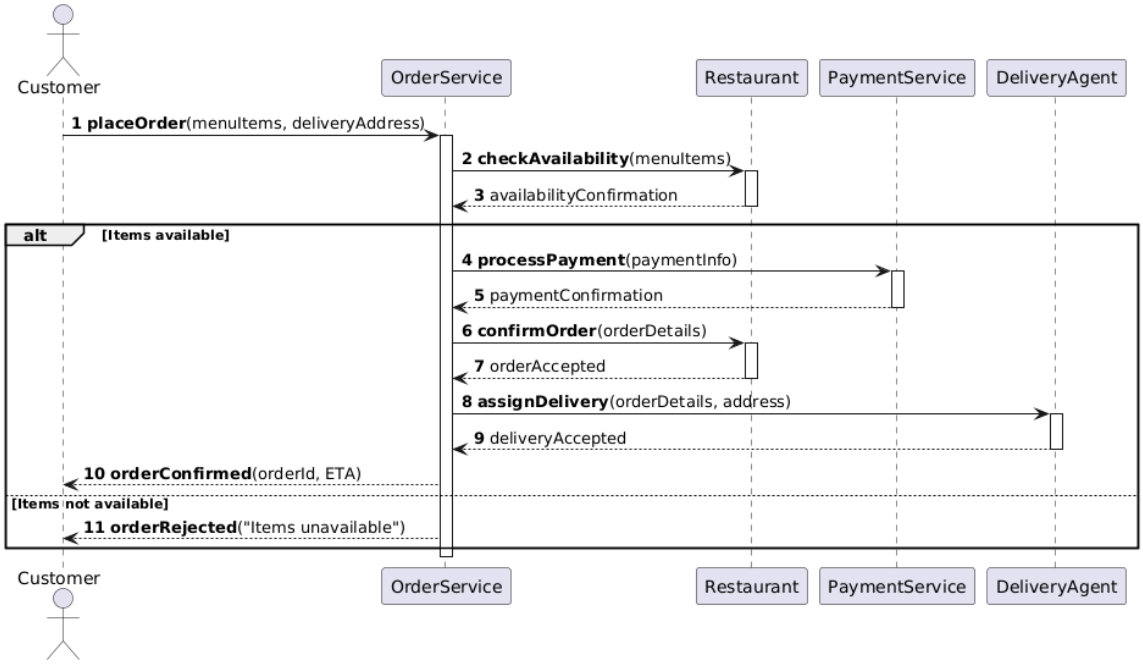

Figure 2: Sequence Diagram of Customer placing a Order

6. Design Model

Class Diagram

- **Inheritance**: Customer, Delivery Agent, and Restaurant inherit from User.
- **Association/Composition**: An Order has multiple FoodItem objects.
- **Encapsulation**: Use private fields and public getters/setters.

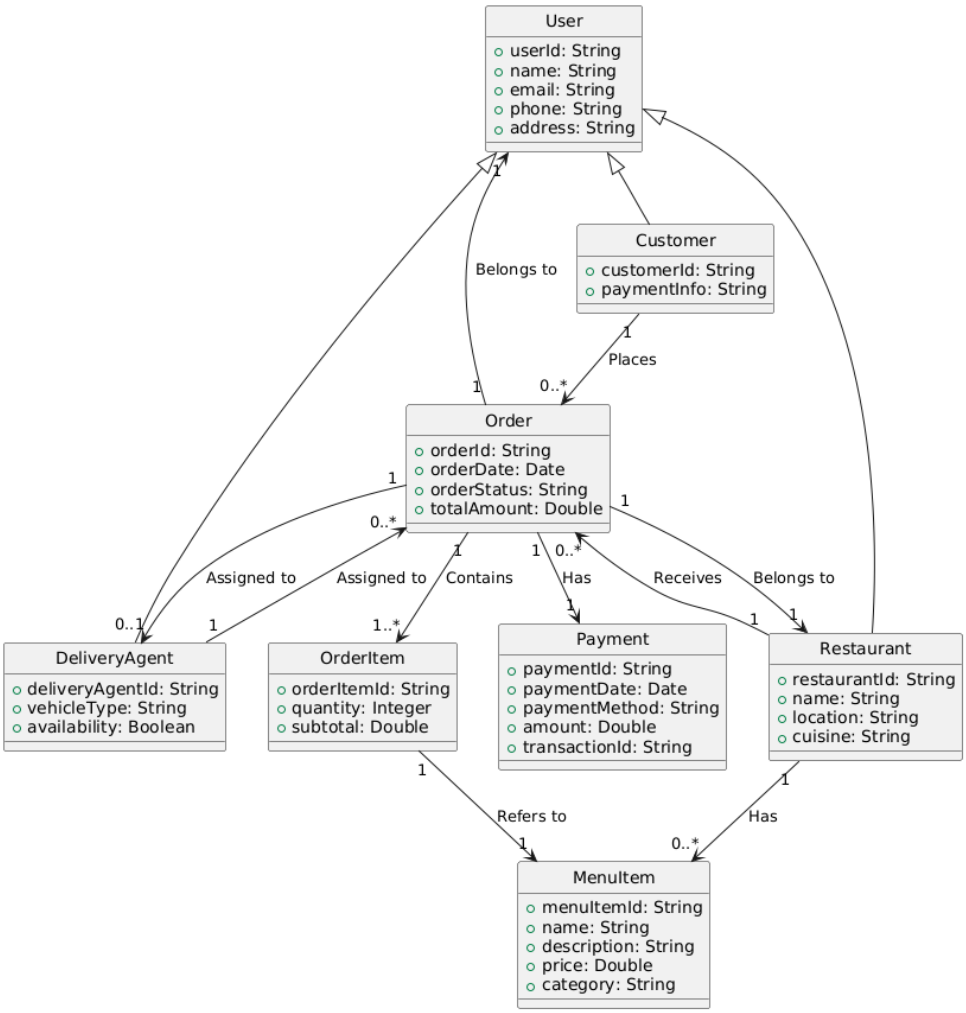

Figure 3: Class Diagram

Figure 3 [2] outlines the food delivery system's core structure. The User class is inherited by Customer, Restaurant, and DeliveryAgent, each with unique attributes. The Order class links to OrderItem (food items), Payment (transactions), and interacts with Restaurant (order source) and DeliveryAgent (fulfillment). Restaurants manage MenuItem objects, while customers place and track orders through these interconnected classes, capturing the full delivery workflow.

7. Implementation Model

The online food delivery system is designed to provide a robust, scalable, and user-friendly platform connecting customers, restaurants, and delivery personnel through a digital medium. The Implementation of the online food delivery system can be done across the following way:

Frontend Development

Mobile Applications:

- Android Platform: Developed using Kotlin.
- iOS Platform: Built with Swift.
- Cross-platform Option: Some services may use React Native or Flutter for cost-efficient development across both platforms

Web Interface:

- Built using React.js or Angular frameworks for dynamic user experiences

Backend Development

- Programming Languages: Node.js with Express.js framework and Python with Django/Flask

API and Third-Party Integrations

Payment Gateways:

- Nepal: Khalti, eSewa, IME Pay integration
- International: Stripe for cross-border payments

Mapping and Location Services:

- Google Maps API for location tracking and route optimization

OOP Implementation Example

Some Sample way OOP concepts can be implemented using python [3] is as follows:

1. Encapsulation

Sample Code:

```
class Restaurant:
    def __init__(self, name, cuisine):
        self.__name = name  # Private attribute
        self.__cuisine = cuisine

    def get_name(self):  # Getter method
```

```
        return self.__name

    def set_cuisine(self, cuisine):  # Setter method
        self.__cuisine = cuisine
```

2. Inheritance

Sample Code:

```
class User:
    def __init__(self, name, email):
        self.name = name
        self.email = email


class Customer(User):  # Inherits from User
    def __init__(self, name, email, loyalty_points=0):
        super().__init__(name, email)
        self.loyalty_points = loyalty_points
```

3. Polymorphism

Sample Code:

```
class Payment:
    def process_payment(self, amount):
        raise NotImplementedError

class CreditCard(Payment):
    def process_payment(self, amount):
        print(f"Processing ${amount} via Credit Card")

class Esewa(Payment):
    def process_payment(self, amount):
        print(f"Processing NPR {amount} via eSewa")
```

4. Abstraction

Sample Code:

```
from abc import ABC, abstractmethod

class OrderService(ABC):
    @abstractmethod
    def place_order(self, items):
        pass

class FoodDeliveryService(OrderService):
    def place_order(self, items):
        print(f"Order placed: {items}")
```

8. Conclusion and Recommendations

The Online Food Delivery System demonstrates how OOP principles can simplify complex real-world problems. By modeling users, orders, and restaurants as objects, the system becomes more intuitive and maintainable. Future improvements could include:

- More robust error handling and exception classes.
- Adding interfaces for delivery tracking.
- Apply SOLID principles for better OOP design.

References

[1] G. Booch, Object-Oriented Analysis and Design with Applications, Addison-Wesley, 2007.

[2] "The Unified Modeling Language," [Online]. Available: https://www.uml-diagrams.org/. [Accessed 17 May 2025].

[3] "Object-Oriented Programming in Python," [Online]. Available: https://python-textbok.readthedocs.io/en/latest/. [Accessed 17 May 2025].